Progress Log (Phase 2) — Weeks 13–20

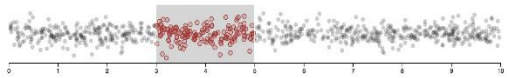
Week 13 — Phase-2 kick-off (Creation focus)

Stage of Project: Start of Phase-2 build, targeting first visuals and baseline UX.

What progress have you made this topic?
This week marked the beginning of Phase 2, and I was able to create the first two visuals: the scatter plot and the radar chart. The scatter plot now has proper CSV parsing, stable hover detection, and an interactive brush embedded directly into the legend, which felt like a good design breakthrough. The radar chart now runs smoothly thanks to caching strategies, and I also introduced intuitive controls like year-pill toggles and a help button. Overall, I established the baseline UX and interaction patterns that later charts will follow.

Tasks Completed:

- **Scatter Plot (bread vs meat, uk-food-spending.js).** Parsed CSV with defensive header detection, built x/y mappers, grid/ticks, and a chronological polyline so temporal flow is visible. Added a draggable year-range in the legend with real-time filtering; hover is computed once per frame for a stable highlight + tooltip.



Legend-embedded brush (inspiration: d3 brush)

- **Radar Chart (monthly rainfall, avg-rainfall.js).** Normalised data to a "year to 12 months" structure, drew a cached radial grid/labels, and overlaid multi-year polygons. When a year is toggled on it eases in above the cache and then merges into a staticLayer. Added year-pill controls and a small "?" help toggle.

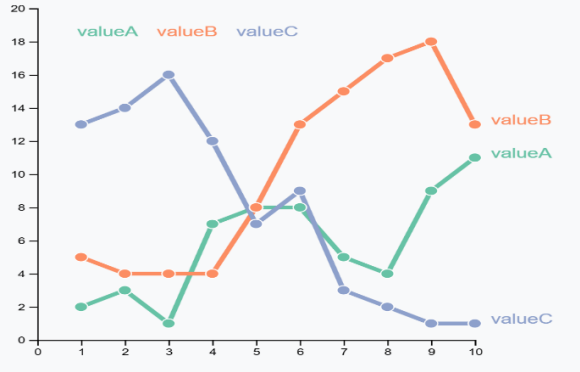
What I Learned: A compact brush can live inside the legend without adding separate UI, provided that drag bounds and labels are clear. Caching static elements (radar grid/labels and already-settled polygons) keeps frame time predictable and leaves headroom for hover and small animations.

What problems have you faced and were you able to solve them?
Two main problems stood out. First, the scatter plot's legend drag conflicted with double-click reset, which made interaction unreliable. I solved this with edge-triggered input and a timing check for double clicks. Second, the CSV dataset had inconsistent year headers, which broke mappings. I fixed it by adding a regex fallback and recalculating the minimum and maximum years. Both issues were resolved without cutting features.

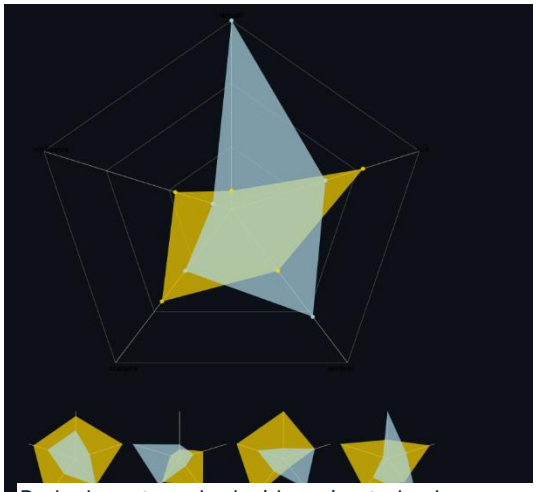
Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?
Yes, I am on target. Phase 2 started smoothly and both charts demonstrate reusable techniques

like caching and embedded UI that will save time in later builds. The only risk is making sure that compare features scale well across multiple charts, but I've already laid the groundwork. If new issues arise, I'll address them by standardizing interaction logic early, so fixes apply across all charts.

Inspiration and Evidence:



Connected Scatter Plot (D3 Graph)



Radar layout: cached grid + animated polygon

Week 14 — Creation + first compare modes

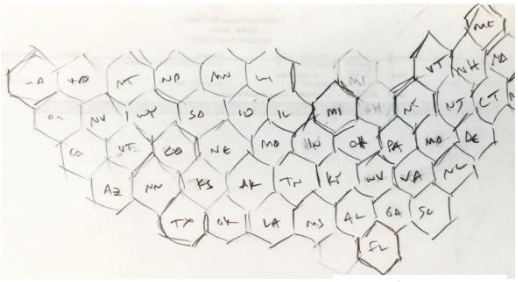
Stage of Project: Second wave of charts; introduce comparison affordances.

What progress have you made this topic?
This week I pushed the project into its second wave of visuals by adding the heatmap and the hex-tile choropleth, both of which introduced the first real comparison features. The heatmap now has a staged row-by-row reveal, pinned year selection, and a Shift-based second year comparison, along with a compact diverging strip that provides a quick visual of differences. The hex map brought in geographical representation using fixed hex coordinates, and I made it interactive with keyboard controls and pinning options. These two charts mark the point where the project moved from single-view charts into true comparative visual analysis.

Tasks Completed:

- **Heatmap (UK monthly inflation, uk-inflation.js)**: Built a month×year grid with staged row-by-row reveal, sequential palette, and pin + Shift-compare (pick a base year, then hold Shift to compare a second year). Added a compact diverging strip that summarizes month-level changes between the two years, plus tooltip clamping to keep labels inside the viewport.

- **Hex-tile choropleth (ethnicity by region, popultaion-by-ethnicity.js):** Laid out a fixed hex tile grid (axial coordinates to polygons), applied quantile binning to color tiles, and wired key controls; ←/→ to switch ethnicity and ↑/↓ to toggle metric ("% of region", "% of ethnic group"). Implemented click-to-pin and Shift-compare with a dashed connector line.



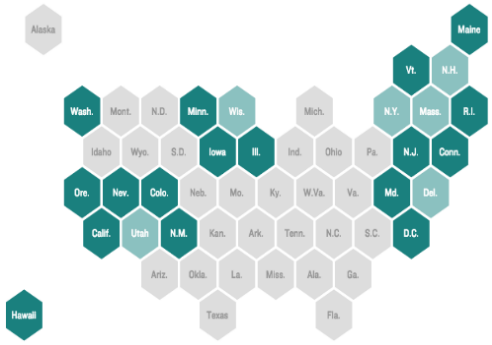
hex grid sketch

What I Learned: A small diverging strip gives "at a glance" directionality without overwhelming the heatmap, and quantile bins make category changes readable across regions when scales differ per metric. Modifier-key selection (Shift for multi-select) feels natural and avoids extra UI controls.

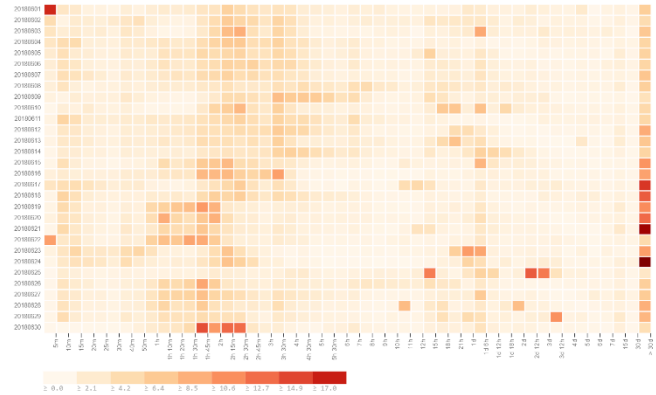
What problems have you faced and were you able to solve them?
I ran into two specific issues. In the heatmap, the diverging strip arrows pointed the wrong direction at first, which made comparisons confusing. I solved this by adjusting the key handling to be edge-triggered and by computing a signed meanDelta. On the hex map, the dashed compare connector "stuck" and carried over to later strokes, which broke the visual consistency. I fixed it by isolating the connector segment and restoring the dash pattern after each draw pass. Both fixes restored clarity and reliability in interactions.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?
Yes, I am on target. Both charts were completed with their compare features functioning, and the technical issues were resolved without major setbacks. The challenge will be ensuring consistency across all charts, but I've already set a clear interaction pattern to follow. If new issues arise, I'll handle them by centralizing shared logic (like compare toggling) so fixes apply everywhere.

Inspiration and Evidence:



Hexagons for tile grid maps (NPR Apps)



Heatmap layout reference (Observable D3 Heatmap example)

Week 15 — Creation + 3D and statistics

Stage of Project: Visual/technical complexity; statistical summaries.

What progress have you made this topic?
This week I made significant progress by adding two technically demanding chart types: a pseudo-3D bar chart and a statistical box plot. The 3D bar chart introduced camera-like transforms, inertia-based rotation, and zoom, giving users a spatial view of regional population and density. The box plot required careful statistical parsing and calculation of quartiles, medians, and fences, while also making it interactive with pinned tooltips and median comparisons. Together, these charts demonstrated that the project could handle both visual complexity and rigorous statistical summaries.

Tasks Completed:

- **3D Bar (regional population & density, regional-population.js):** Implemented 2D-canvas transforms for a pseudo-3D view: grid, column bases, and extruded bars drawn with ordered faces. Added orbit drag with inertia and zoom, plus a reveal-from-ground animation. To keep labels legible, faces are painted back-to-front (painter's-algorithm style) and occluded labels fade at steeper angles.

- **Box Plot (vehicle age by impact, vehicle-age.js):** Wrote a robust parser for messy inputs (single values, ranges like "3–5", and code values to ignore). Computed Q1/Median/Q3, IQR, and Tukey fences; drew boxes/whiskers, jittered points, and orbiting outliers to keep them visible but unobtrusive. Added pin tooltip and Shift-compare medians with a label anchored to the true medians.

What I Learned: For pseudo-3D on a 2D canvas, clean results come from orderly transforms (translate/rotate/applyMatrix) and a simple painter's-algorithm pass rather than heavyweight depth buffers.

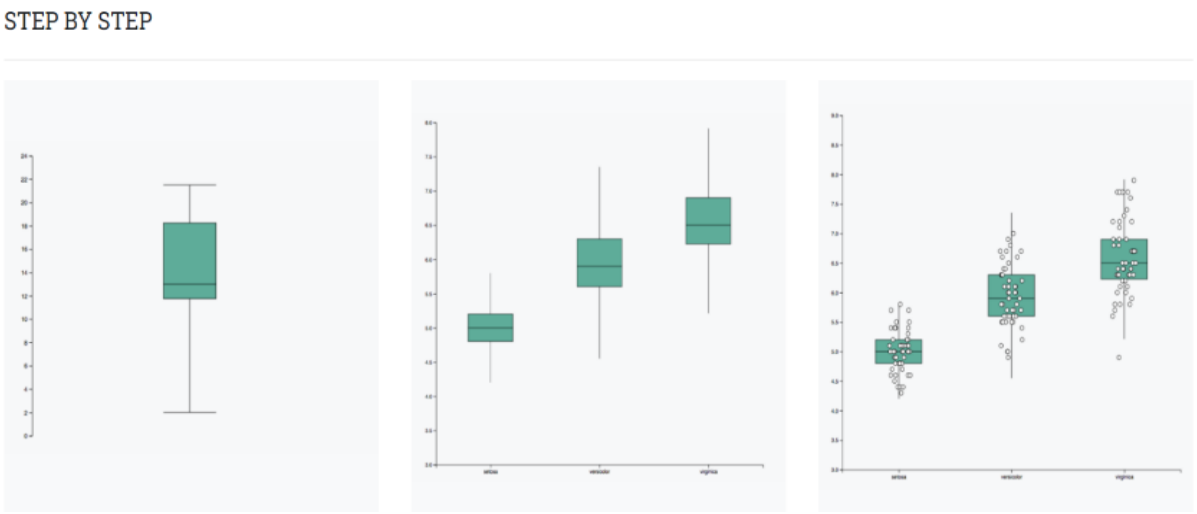
What problems have you faced and were you able to solve them?

The main problem with the 3D bar chart was drag inertia: at first, the bars spun endlessly after interaction, which broke usability. I solved this by tuning damping and easing the deltas for smoother, more controlled motion. In the box plot, inconsistent data (like ranges and coded values) corrupted the quartile calculations. I wrote a dedicated parser to sanitize input values before computing statistics, which fixed the issue.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

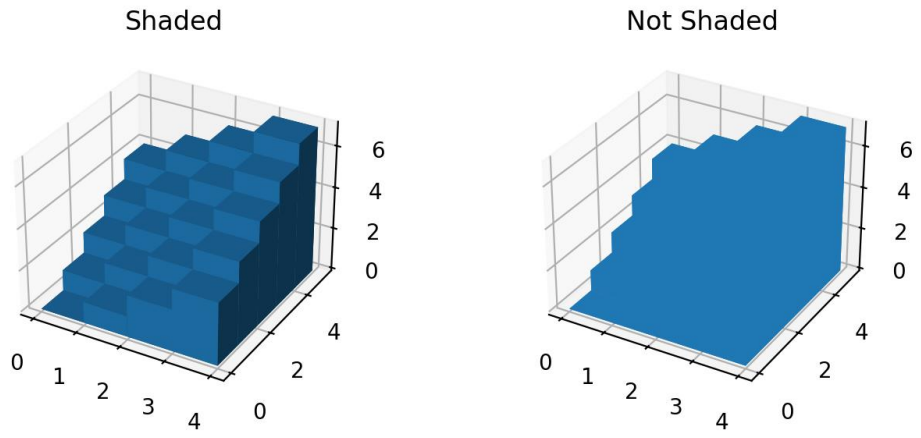
Yes, I am on target. Both charts reached a working, interactive state with their technical hurdles resolved. These were some of the hardest charts to implement, and completing them on schedule gave me confidence about the remaining Phase-2 workload. If new challenges come up, I'll rely on the same approach of breaking down problems (transform order, parsing, edge cases) and resolving them systematically.

Inspiration and Evidence:



Pattern inventory for box-and-whisker construction; adapted to p5 (D3 Graph Gallery)

Demo of 3D bars (Matplotlib)



- ❖ Painter's Algorithm (Wikipedia): Back-to-front face painting used for bar occlusion ordering.
- ❖ p5.js; applyMatrix() / rotate(): Coordinate transforms used for orbit/tilt

Week 16 — Creation + large-data pipeline

Stage of Project: Final two visuals; streaming + caching.

What progress have you made this topic?
This week I completed the final two major visuals of Phase 2: the streamgraph and the chord diagram. The streamgraph introduced advanced stacking logic with wiggle/inside-out ordering, dual-pane comparison (absolute vs normalized), and animated "steam/wobble" effects that make the chart feel alive. The chord diagram pushed the project into large-data handling, using a streaming CSV parser to process files incrementally and map them into adjacency matrices for arc-and-ribbon layouts. Both charts worked as intended, proving the framework can handle both continuous time-series and complex relational data.

Tasks Completed:

- **Streamgraph (UK renewables, uk-renewables.js):** Implemented inside-out ordering and the wiggle/symmetric baseline so layers undulate around a central axis. Added a left pane for absolute values and a right pane for Compare with Normalize to % (per-pane scales so modes don't bleed). Introduced a subtle "steam/wobble" effect driven by a timer so large bands feel alive.
- **Chord diagram (UK–US fashion flows, uk-us-fashion.js):** Built a streaming CSV parser (Fetch ReadableStream + TextDecoder), buffering partial lines and assembling rows

chunk-by-chunk without freezing the UI; mapped the cleaned table into n×n matrix for the chord layout, with Top-N + “Other” aggregation and a localStorage cache keyed by dimensions to avoid rebuilds. Added a progress indicator and a cancel token.

What I Learned: For streamgraphs, offset=wiggle + inside-out order yields the most legible band motion (less baseline drift) and is the standard recommendation. For chords, treating the dataset as an adjacency matrix is the best and easiest approach to the chart.

What problems have you faced and were you able to solve them?
The streamgraph had category toggling issues where bands jittered because the envelopes rebuilt without a stable sort. I solved this by enforcing deterministic inside-out ordering and only rebuilding once per toggle. The chord diagram faced problems with stale builds when the stream reader left partial lines unassembled. I wrote a small state machine to handle incomplete chunks and introduced a cancelable reader to ensure that old builds stopped cleanly. These fixes made both visuals stable and responsive.

Are you on target to successfully complete your project? If you aren’t on target, how will you address the issue?
Yes, I am on target. These were the final two complex visuals, and completing them on time means I’m ahead of schedule for the polish and refinement stages. If any new issues appear, I’ll address them through targeted testing and by reusing the caching and streaming strategies that already proved effective here.

Inspiration and Evidence:

Reference logic for wiggle/inside-out comes from Byron–Wattenberg's geometry paper.

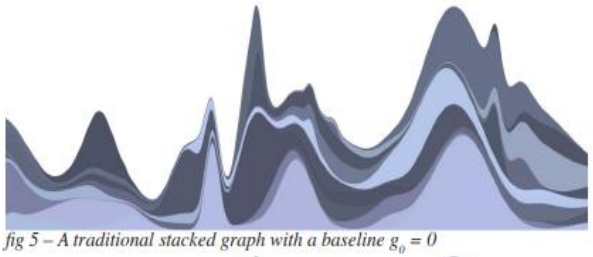


fig 5 – A traditional stacked graph with a baseline $g_0 = 0$

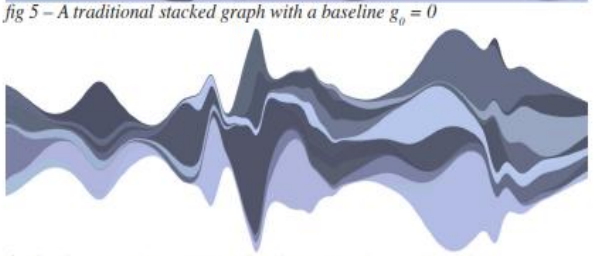


fig 6 – the same data set using the ThemeRiver layout algorithm

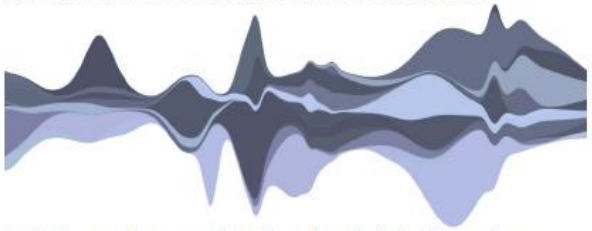


fig 7 – the same data set optimized to reduce the "wiggle" function, or overall variation in slope

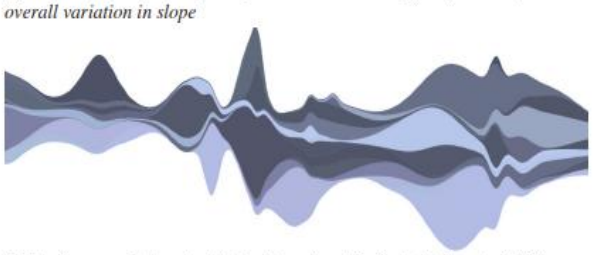


fig 8 – the same data set optimized to reduce the "weighted_wiggle," the algorithm used in Streamgraph

fig 12 – an unsorted data set, exhibiting the type of "burstiness" apparent in last.fm and box office data sets

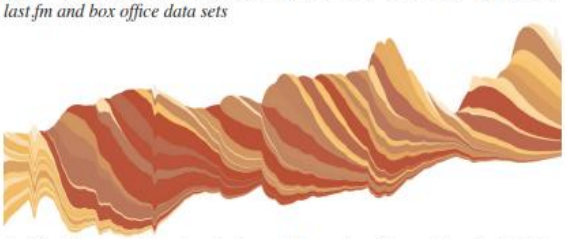


fig 13 – the same data set, naively sorted in order of "onset time" exhibiting the distracting diagonal striping effect

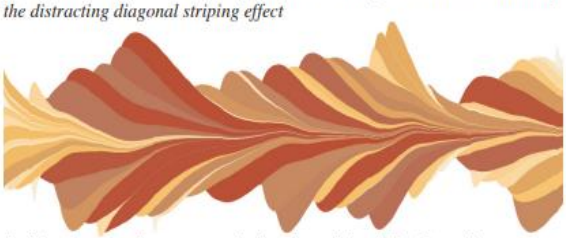


fig 14 – the same data set sorted using the weighted "inside out" strategy to highlight the initial onset of each time series

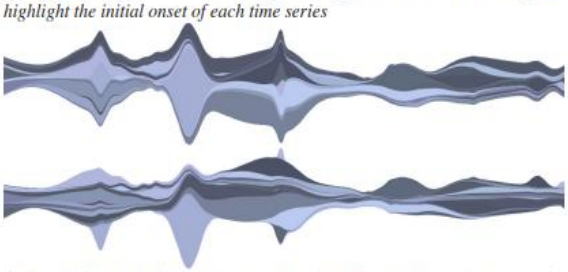
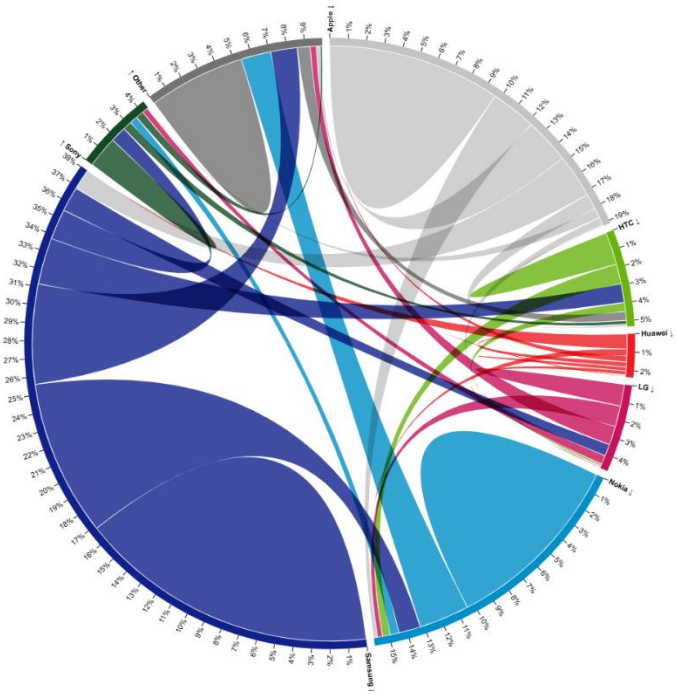


fig 15 – before and after applying an "inside out" sort using a "measure of volatility" in place of "onset time"

- Streaming CSV: MDN; ReadableStream.getReader() and TextDecoderStream



Reference look & feel for arcs + ribbons + labels. (Observable)

Week 17 — Refinement: accessibility, guidance, consistency

Stage of Project: Refinement: transitions, keyboard discoverability, readable helpers, and consistent legends.

What progress have you made this topic?
This week I shifted focus from creating new visuals to refining the usability and accessibility of the project. I introduced on-canvas microcopy that provides instant context (e.g., “Shift=Compare, ESC=Clear”), standardized legend formatting across all charts, and improved tooltip contrast and accessibility. These changes ensure that interactions are more discoverable, legends look consistent, and tooltips remain readable even on busy or high-contrast backgrounds.

Tasks Completed: Added on-canvas microcopy for expert shortcuts (e.g., “Shift=Compare, ESC=Clear”) where compare exists; standardized legend spacing, font sizes, tooltip contrast checks, and key order across charts; introduced focus highlights.

What I Learned: For readability, WCAG AA contrast (≥4.5:1) is a simple, defensible rule for small text; and when a tooltip is HTML, following the ARIA tooltip pattern keeps it accessible by hover and focus.

What problems have you faced and were you able to solve them?

Two main problems emerged. First, legends across different charts drifted in both order and color mapping, which created confusion for users. I solved this by centralizing the palette mapping and ensuring all legends are generated from the same data order. Second, small text sometimes failed contrast checks, especially on strong colors like the heatmap's reds or the streamgraph's darker bands. I addressed this by darkening grey tones and adding a subtle 1 px halo around labels in hotspots to pass WCAG AA standards.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?
Yes, I am on target. By addressing legend consistency and contrast issues now, I've removed a common source of user confusion and ensured the visuals meet accessibility guidelines. The project is moving into its final polish stages, and I'm confident that remaining refinements will stay on schedule.

Inspiration and Evidence:

- NN/g; UX Copy Sizes: Long, Short, and Micro: Microcopy: short, context help placed where the action happens.
- WAI-ARIA Authoring Practices: Tooltip and MDN: ARIA: tooltip role

Week 18 — Performance & documentation

Stage of Project: De-jank week: profile the visuals, tighten animation timing, and evidence for report.

What progress have you made this topic?
This week was dedicated to performance and cleanup rather than adding new visuals. I profiled the project to find bottlenecks, introduced caching for static elements like the radar grid and choropleth lattice, and throttled redraws during interactions such as dragging. I also consolidated helper functions to reduce redundancy and captured annotated screenshots and diagrams for the progress log, which will serve as evidence in the report.

What problems have you faced and were you able to solve them?
The main issue was tiny but noticeable hiccups during fast interactions, especially when dragging legends or hovering rapidly across points. This happened because labels and ticks were being recomputed every frame. I solved it by precomputing all tick and label strings ahead of time, which smoothed out performance without changing visual output.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?
Yes, I am on target. By removing small performance hiccups and documenting my process with

clear screenshots and diagrams, I've strengthened both the technical and reporting sides of the project. If any additional performance issues appear, I'll address them by extending the caching and throttling methods I've already proven effective here.

Week 19 — QA & peer testing

Stage of Project. Cross-device checks; bug-fixing.
Tasks Completed: Ran peer tests of compare flows (Heatmap/Hex/Box); confirmed Shift=Compare and ESC=Clear; audited tooltip bounds; added NaN guards; verified streamgraph Normalize to % sums ≈100% per year; re-centered box-plot median; validated chord Top-N + "Other" against cache/cancel token to prevent stale rebuilds; harmonized legend keys and naming.

What problems have you faced and were you able to solve them?
The main issue was with the chord diagram. Its Top-N aggregation sometimes created very narrow arcs that squeezed labels into unreadable spaces. I fixed this by enforcing a minimum span for arcs and truncating overly long labels so they fit cleanly. This small but important fix made the chord diagram much more usable.

What are you planning to do over the next few weeks?
The plan moving forward is to carry these QA fixes into the final polish. That includes making sure compare semantics, legends, and tooltips remain consistent and reliable across all visuals. I'll also conduct more peer tests to confirm that the fixes hold up under different workflows and edge cases.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?
Yes, I am on target. Peer testing revealed issues, but they were solvable with logic-level fixes that apply broadly across the charts. Since the problems were addressed systematically, I feel confident that the visuals are stable and ready for the final polish stage. If further issues come up, I'll apply the same testing and refinement cycle to resolve them quickly.

Week 20 — Final polish & hand-in prep

Stage of Project: Tie-off aligned with end-window on Gantt timeline (final report prep).
Tasks Completed: Pass over copy/labels, axis tick consistency, legend keys; cleaned code comments; compiled progress log and screenshots.